

Comparative Study of Optimizers for Backpropagation Algorithms

Programming Assignment III

CS601T: Deep Learning

Group 30

Submitted by

Anushaa B (CS24BT032)

Skanda P (CS24BT045)

Thiaana B (CS24BT031)

Under the Supervision of

Prof. Dileep AD



Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

September 2026

Abstract

This report presents a comparative study of seven optimization methods for training fully connected neural networks on a five-class subset of the MNIST handwritten digit dataset. Nine architectures with three, four, and five hidden layers are evaluated using stochastic gradient descent (SGD), batch gradient descent (BGD), SGD with momentum, Nesterov accelerated gradient (NAG), AdaGrad, RMSProp, and Adam. The hidden layers use the hyperbolic tangent (tanh) activation function and the models are trained using cross-entropy loss. All optimizers corresponding to a given architecture start from the same initial weights to ensure a controlled comparison. The results show that momentum-based methods provide the strongest combination of convergence speed and validation performance. The highest validation accuracy is 98.39%, obtained by both 4L-v3 and 5L-v3 with SGD with momentum. The 4L-v3 model is selected for final evaluation and achieves 98.02% test accuracy and 98.02% macro F1-score.

Contents

1	Introduction	1
2	Proposed Methodology	3
2.1	Fully Connected Neural Network	3
2.2	Cross-Entropy Loss	5
2.3	Gradient Descent Optimizers	6
2.3.1	Batch Gradient Descent	6
2.3.2	Stochastic Gradient Descent	6
2.3.3	SGD with Momentum	6
2.3.4	Nesterov Accelerated Gradient	7
2.3.5	AdaGrad	8
2.3.6	RMSProp	8
2.3.7	Adam	9
2.4	Convergence Criterion	10
3	Experimental Setup	11
3.1	Dataset	11
3.2	Network Architectures	12
3.3	Optimizer Configuration	12
3.4	Weight Initialization	13
3.5	Training Procedure	15
3.6	Evaluation Metrics	15

4	Results	17
4.1	Convergence Epochs	17
4.2	Training and Validation Accuracy	18
4.3	Aggregate Optimizer Comparison	19
4.4	Training-error Curves	20
4.5	Effect of Depth and Width	25
5	Best Architecture and Test Evaluation	26
5.1	Model Selection	26
5.2	Selected Model Performance	27
5.3	Selected Model Training Curve	28
5.4	Test-set Performance	29
5.5	Test Confusion Matrix	30
5.6	Training Confusion Matrix	31
5.7	Per-class Test Performance	31
6	Conclusion	32
	References	34

List of Figures

4.1	Superimposed average training cross-entropy loss for 3L-v1. . .	20
4.2	Superimposed average training cross-entropy loss for 3L-v2. . .	21
4.3	Superimposed average training cross-entropy loss for 3L-v3. . .	21
4.4	Superimposed average training cross-entropy loss for 4L-v1. . .	22
4.5	Superimposed average training cross-entropy loss for 4L-v2. . .	22
4.6	Superimposed average training cross-entropy loss for 4L-v3. . .	23
4.7	Superimposed average training cross-entropy loss for 5L-v1. . .	23
4.8	Superimposed average training cross-entropy loss for 5L-v2. . .	24
4.9	Superimposed average training cross-entropy loss for 5L-v3. . .	24
5.1	Optimizer comparison for the selected 4L-v3 architecture. . . .	28
5.2	Test confusion matrix for the selected 4L-v3 model. Rows represent true classes and columns represent predicted classes in the order 3, 4, 5, 7, and 8.	30
5.3	Training confusion matrix for the selected 4L-v3 model.	31

List of Tables

3.1	FCNN architectures used in the experiments.	12
3.2	Optimizer configurations used in the experiments.	13
4.1	Number of epochs required for convergence.	17
4.2	Training accuracy (%) for all architecture–optimizer combinations.	18
4.3	Validation accuracy (%) for all architecture–optimizer combinations.	18
4.4	Aggregate optimizer performance across the nine architectures.	19
5.1	Performance of the selected 4L-v3 model.	27
5.2	Per-class test performance of the selected model.	32

Chapter 1

Introduction

Fully connected neural networks (FCNNs) are feed-forward neural networks in which every neuron in one layer is connected to every neuron in the subsequent layer. An FCNN can represent nonlinear decision boundaries through the use of nonlinear activation functions and multiple hidden layers. The performance of such a network depends on both its architecture and the optimization method used during training.

Training an FCNN using backpropagation requires the gradient of the loss function with respect to the network parameters to be computed and used for parameter updates. Let $\mathbf{w}$ denote the complete vector of trainable parameters and let $L(\mathbf{w})$ denote the loss function. The basic gradient-descent update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_t),$$

where t denotes the current update step, η denotes the learning rate, and $\nabla_{\mathbf{w}} L(\mathbf{w}_t)$ denotes the gradient of the loss with respect to $\mathbf{w}$ at step t .

Although this update provides a direct method for minimizing the loss, the convergence of gradient descent can be slow. Several optimization methods have therefore been developed to improve the behaviour of backpropagation. Momentum-based methods incorporate information from previous updates, while adaptive methods modify the effective learning rate using

information from previous gradients [1].

The objective of this assignment is to compare the convergence behaviour and classification performance of different optimization methods across multiple FCNN architectures. Nine architectures are evaluated using seven optimizers under fixed experimental conditions. The comparison is based primarily on convergence epochs and validation accuracy, followed by evaluation of the selected model on the held-out test set.

The study specifically examines the effect of optimizer choice, network depth, and hidden-layer width on the resulting classification performance.

Chapter 2

Proposed Methodology

2.1 Fully Connected Neural Network

Each input image has dimensions 28×28 pixels and is therefore represented by 784 input features after flattening. Let the input vector for sample n be

$$\mathbf{x}_n \in R^{784},$$

where n denotes the sample index.

Consider a network containing L hidden layers. Let n_l denote the number of neurons in hidden layer l , where $l = 1, \dots, L$. Let $\mathbf{a}_n^{(l-1)}$ denote the activation vector entering hidden layer l . For the input layer,

$$\mathbf{a}_n^{(0)} = \mathbf{x}_n,$$

where

$$\mathbf{a}_n^{(0)} \in R^{784}.$$

For hidden layer l , let

$$\mathbf{W}^{(l)} \in R^{n_l \times n_{l-1}}$$

denote the weight matrix and let

$$\mathbf{b}^{(l)} \in R^{n_l}$$

denote the bias vector. The pre-activation vector is

$$\mathbf{z}_n^{(l)} = \mathbf{W}^{(l)} \mathbf{a}_n^{(l-1)} + \mathbf{b}^{(l)},$$

where

$$\mathbf{z}_n^{(l)} \in R^{n_l}.$$

The activation vector of the hidden layer is

$$\mathbf{a}_n^{(l)} = \tanh(\mathbf{z}_n^{(l)}),$$

where the hyperbolic tangent is applied element-wise.

The tanh activation function is

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$

and its derivative is

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z).$$

The output layer contains $K = 5$ neurons corresponding to the five digit classes $\{3, 4, 5, 7, 8\}$. Let

$$\mathbf{W}^{(L+1)} \in R^{5 \times n_L}$$

denote the output-layer weight matrix and let

$$\mathbf{b}^{(L+1)} \in R^5$$

denote the output-layer bias vector.

The output logits for sample n are therefore

$$\mathbf{z}_n^{(L+1)} = \mathbf{W}^{(L+1)} \mathbf{a}_n^{(L)} + \mathbf{b}^{(L+1)},$$

where

$$\mathbf{z}_n^{(L+1)} \in R^5.$$

Thus, the output layer produces five logits, one corresponding to each class. The hidden layers use tanh activation for all experiments reported in this study.

2.2 Cross-Entropy Loss

Let N denote the number of training samples and let $K = 5$ denote the number of classes. For sample n , let $y_n \in \{1, \dots, K\}$ denote its true class index and let z_{nk} denote the logit corresponding to class k .

The softmax probability assigned to class k is

$$p_{nk} = \frac{\exp(z_{nk})}{\sum_{j=1}^K \exp(z_{nj})},$$

where j is the class-index variable.

The cross-entropy loss for sample n is

$$L_n = -\log p_{n,y_n}.$$

The average training loss over the N training samples is

$$L = -\frac{1}{N} \sum_{n=1}^N \log p_{n,y_n}.$$

The implementation applies cross-entropy loss directly to the output logits. Consequently, a separate softmax layer is not included in the FCNN.

2.3 Gradient Descent Optimizers

2.3.1 Batch Gradient Descent

Let $\mathbf{w}_t$ denote the complete parameter vector at update step t , and let η denote the learning rate. Batch gradient descent computes the gradient using the complete training set before performing an update [1]:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_t).$$

Thus, one parameter update is performed after processing all N training samples.

2.3.2 Stochastic Gradient Descent

In stochastic gradient descent (SGD), one training sample is used for each parameter update. Let $L_n(\mathbf{w}_t)$ denote the loss for training sample n at update step t . The update is [1]

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L_n(\mathbf{w}_t).$$

For the experiments, the batch size for SGD is 1. Therefore, each training sample produces one parameter update.

2.3.3 SGD with Momentum

Momentum incorporates the previous parameter change into the current update [3]. Let $\Delta \mathbf{w}_t$ denote the parameter change at update step t and let α denote the momentum coefficient. The update is

$$\Delta \mathbf{w}_t = -\eta \nabla_{\mathbf{w}} L_t + \alpha \Delta \mathbf{w}_{t-1},$$

followed by

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}_t.$$

Here, L_t denotes the loss associated with the current parameter update. The parameter α controls the contribution of the previous parameter change. The experiments use

$$\eta = 0.001, \quad \alpha = 0.9.$$

The use of previous parameter changes allows the optimizer to maintain progress when successive gradients favour similar directions.

2.3.4 Nesterov Accelerated Gradient

Nesterov accelerated gradient (NAG) also incorporates the previous parameter change but evaluates the gradient at a look-ahead point [2].

Let $\Delta \mathbf{w}_{t-1}$ denote the parameter change from the previous update and let α denote the momentum coefficient. The look-ahead parameter vector is

$$\mathbf{w}_{\text{lookahead}} = \mathbf{w}_t + \alpha \Delta \mathbf{w}_{t-1}.$$

The parameter change is then computed as

$$\Delta \mathbf{w}_t = \alpha \Delta \mathbf{w}_{t-1} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_{\text{lookahead}}),$$

followed by

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}_t.$$

The experiments use

$$\eta = 0.001, \quad \alpha = 0.9.$$

The look-ahead evaluation allows the gradient to provide information about the direction of the forthcoming update.

2.3.5 AdaGrad

AdaGrad assigns an adaptive effective learning rate to each parameter [4]. Let w_l denote parameter l , let $g_l(t)$ denote the gradient of the loss with respect to w_l at update step t , and let $v_l(t)$ denote the accumulated squared-gradient value.

The accumulated value is

$$v_l(t) = v_l(t-1) + g_l(t)^2.$$

The corresponding parameter update is

$$w_l(t+1) = w_l(t) - \frac{\eta}{\sqrt{v_l(t)} + \epsilon} g_l(t),$$

where η is the learning rate and ϵ is a small positive constant used for numerical stability.

The experiments use

$$\eta = 0.001, \quad \epsilon = 10^{-8}.$$

Because the squared-gradient history continually accumulates, the effective learning rate can decrease substantially over time.

2.3.6 RMSProp

RMSProp replaces the cumulative squared-gradient history used by AdaGrad with an exponentially weighted moving average [5].

Let $v_l(t)$ denote the moving average of squared gradients for parameter w_l , let $g_l(t)$ denote the corresponding gradient, and let β denote the decay coefficient. The moving average is

$$v_l(t) = \beta v_l(t-1) + (1 - \beta) g_l(t)^2.$$

The parameter update is

$$w_l(t+1) = w_l(t) - \frac{\eta}{\sqrt{v_l(t)} + \epsilon} g_l(t).$$

The experiments use

$$\eta = 0.001, \quad \beta = 0.99, \quad \epsilon = 10^{-8}.$$

The exponentially weighted history prevents the denominator from increasing indefinitely, thereby reducing the aggressive learning-rate decay associated with AdaGrad.

2.3.7 Adam

Adam combines a moving average of the gradient with a moving average of the squared gradient [6].

Let $g_l(t)$ denote the gradient of the loss with respect to parameter w_l at update step t . Let $u_l(t)$ denote the first-moment estimate and let $v_l(t)$ denote the second-moment estimate. The first moment is

$$u_l(t) = \beta_1 u_l(t-1) + (1 - \beta_1) g_l(t),$$

and the second moment is

$$v_l(t) = \beta_2 v_l(t-1) + (1 - \beta_2) g_l(t)^2.$$

Since both moment estimates are initialized at zero, bias correction is applied:

$$\hat{u}_l(t) = \frac{u_l(t)}{1 - \beta_1^t},$$

$$\hat{v}_l(t) = \frac{v_l(t)}{1 - \beta_2^t}.$$

The parameter update is

$$w_l(t+1) = w_l(t) - \eta \frac{\hat{u}_l(t)}{\sqrt{\hat{v}_l(t)} + \epsilon}.$$

The experiments use

$$\eta = 0.001, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

2.4 Convergence Criterion

Let L_t denote the average training cross-entropy loss measured after epoch t . Training is terminated when the absolute difference between the losses of two consecutive epochs becomes smaller than the specified threshold:

$$|L_t - L_{t-1}| < 10^{-4}.$$

Thus, convergence is determined by the change in average training loss rather than by a fixed number of epochs.

Chapter 3

Experimental Setup

3.1 Dataset

The experiments use the dataset assigned to Group 30, consisting of a five-class subset of the MNIST handwritten digit dataset. The five classes are

$$\{3, 4, 5, 7, 8\}.$$

Each sample is a grayscale image of dimensions 28×28 pixels. After flattening, each image is represented by a vector

$$\mathbf{x}_n \in R^{784}.$$

Each class contains 2,277 training samples, 759 validation samples, and 759 test samples. Therefore, the three splits contain 11,385 training samples, 3,795 validation samples, and 3,795 test samples. The five classes are balanced within each split.

3.2 Network Architectures

Nine FCNN architectures were evaluated. The architectures contain three, four, or five hidden layers, with three width configurations at each depth. The output layer contains five neurons, one for each class.

Table 3.1: FCNN architectures used in the experiments.

Architecture	Hidden-layer widths	Trainable parameters
3L-v1	512–256–128	566,789
3L-v2	256–128–64	242,437
3L-v3	256–256–256	333,829
4L-v1	512–256–128–64	574,725
4L-v2	256–128–64–32	244,357
4L-v3	384–256–128–64	441,477
5L-v1	512–384–256–128–64	738,949
5L-v2	256–256–128–128–64	324,741
5L-v3	256–128–64–64–32	248,517

For example, the 4L-v3 architecture has four hidden layers with 384, 256, 128, and 64 neurons, respectively. Its complete layer configuration is therefore

$$784 \rightarrow 384 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 5.$$

3.3 Optimizer Configuration

All seven optimizers were evaluated for each of the nine architectures, resulting in

$$9 \times 7 = 63$$

architecture–optimizer combinations.

The optimizer configurations are given in Table 3.2.

Table 3.2: Optimizer configurations used in the experiments.

Optimizer	Batch size	Learning rate	Additional parameters	ϵ
SGD	1	0.001	–	–
BGD	N	0.001	–	–
SGD + Momentum	1	0.001	$\alpha = 0.9$	–
SGD + NAG	1	0.001	$\alpha = 0.9$	–
AdaGrad	N	0.001	–	10^{-8}
RMSProp	N	0.001	$\beta = 0.99$	10^{-8}
Adam	1	0.001	$\beta_1 = 0.9, \beta_2 = 0.999$	10^{-8}

Here, $N = 11,385$ denotes the number of training samples. Thus, a batch size of N corresponds to full-batch training.

3.4 Weight Initialization

For each architecture, a single fixed initial state is generated using a fixed random seed and reused for all seven optimizers. The weights use Xavier-normal initialization, while the biases are initialized to zero.

Xavier (Glorot) Normal Initialization

Xavier initialization scales the initial weights according to the sizes of the connected layers. It aims to limit the growth or shrinkage of activations and gradients through the network, rather than guaranteeing that vanishing or exploding gradients cannot occur. This is particularly relevant for the zero-centred tanh activation used in this study.

For a layer with n_{in} input units (fan-in) and n_{out} output units (fan-out), each weight is sampled independently from a zero-mean Gaussian distribution:

$$W_{ij} \sim \mathcal{N}(0, \sigma^2), \quad \sigma = g \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}},$$

where g is the gain factor. Consequently,

$$\text{Var}(W_{ij}) = \frac{2g^2}{n_{\text{in}} + n_{\text{out}}}.$$

This initialization uses $g = 1$, giving variance $2/(n_{\text{in}} + n_{\text{out}})$. This is the default gain of PyTorch’s `xavier_normal_` initializer.

Bias Initialization

All bias parameters are initialized to zero:

$$b_j = 0, \quad j = 1, \dots, n_{\text{out}}.$$

The independently sampled weights distinguish the neurons at initialization, so setting the biases to zero does not make their initial parameter configurations identical.

Example: First Two Hidden Layers of 3L-v1

The 3L-v1 architecture has hidden-layer widths [512, 256, 128]. Its first hidden layer receives the flattened 784-dimensional image, so

$$\mathbf{W}^{(1)} \in R^{512 \times 784}, \quad \mathbf{b}^{(1)} \in R^{512}.$$

With $g = 1$, the standard deviation is

$$\sigma_1 = \sqrt{\frac{2}{784 + 512}} = \sqrt{\frac{1}{648}} \approx 0.0392837.$$

Thus, each entry of $\mathbf{W}^{(1)}$ is drawn independently from $\mathcal{N}(0, 0.0392837^2)$, and every entry of $\mathbf{b}^{(1)}$ is zero. For the next layer, connecting 512 units to 256 units,

$$\sigma_2 = \sqrt{\frac{2}{512 + 256}} = \sqrt{\frac{2}{768}} \approx 0.0510310.$$

The same fan-in/fan-out rule is applied to the remaining layers.

Before each optimizer is trained, the corresponding initial state is restored. Consequently, the seven optimizer runs associated with a given architecture begin from exactly the same point in parameter space. This controls

for initialization when comparing convergence and validation performance. For example, all seven runs of the 3L-v1 architecture begin with an initial training loss of 1.682995.

3.5 Training Procedure

For SGD, SGD with momentum, NAG, and Adam, the training samples are randomly permuted at the beginning of each epoch and the parameters are updated after every sample. These methods therefore use batch size 1.

For BGD, AdaGrad, and RMSProp, the complete set of $N = 11,385$ training samples is used for each parameter update.

The average training loss is recorded once after each epoch for all optimizers.

Algorithm 1 Training procedure for one architecture–optimizer pair

- 1: Load the fixed initial weights for the selected architecture.
 - 2: Initialize the optimizer state.
 - 3: Compute the initial average training loss.
 - 4: **for** $t = 1, 2, \dots$ **do**
 - 5: Train for one epoch using the specified batch size.
 - 6: Compute the average training loss L_t .
 - 7: **if** $t > 1$ and $|L_t - L_{t-1}| < 10^{-4}$ **then**
 - 8: Stop training.
 - 9: **end if**
 - 10: **end for**
 - 11: Evaluate training and validation performance.
-

3.6 Evaluation Metrics

Training accuracy and validation accuracy are used as the primary classification measures. Convergence epochs are used to compare optimization

speed.

For the final selected model, test accuracy, macro precision, macro recall, and macro F1-score are reported. The confusion matrix is also used to examine class-specific classification errors.

For a given class k , let TP_k , FP_k , and FN_k denote the true-positive, false-positive, and false-negative counts for that class. Precision is

$$P_k = \frac{TP_k}{TP_k + FP_k},$$

and recall is

$$R_k = \frac{TP_k}{TP_k + FN_k}.$$

The class-wise F1-score is

$$F1_k = \frac{2P_kR_k}{P_k + R_k}.$$

Macro F1-score is obtained by averaging the F1-scores across the $K = 5$ classes, giving equal importance to each class.

Chapter 4

Results

4.1 Convergence Epochs

Table 4.1 gives the number of epochs required by each architecture–optimizer combination to satisfy the convergence criterion.

Table 4.1: Number of epochs required for convergence.

Architecture	SGD	BGD	Momentum	NAG	AdaGrad	RMSProp	Adam
3L-v1	45	1722	14	15	652	47	23
3L-v2	44	1964	15	15	640	122	24
3L-v3	42	1983	14	15	411	165	47
4L-v1	34	1841	16	15	659	68	88
4L-v2	35	2141	19	21	753	157	74
4L-v3	35	1759	17	20	665	170	52
5L-v1	31	1634	11	22	197	61	79
5L-v2	21	1862	19	18	253	325	125
5L-v3	33	2312	23	24	809	45	70

A substantial difference is observed between BGD and the momentum-based methods. BGD requires between 1,634 and 2,312 epochs across the nine architectures. In comparison, SGD with momentum converges in 11–19 epochs and NAG converges in 15–24 epochs.

Vanilla SGD requires between 21 and 45 epochs. AdaGrad requires hundreds of epochs for most architectures, while RMSProp and Adam exhibit more architecture-dependent behaviour.

The substantially smaller convergence counts of the momentum-based methods are consistent with their use of previous parameter updates. Rather than relying exclusively on the current gradient, momentum incorporates the direction of the preceding update, allowing continued progress when successive updates favour similar directions.

4.2 Training and Validation Accuracy

The training accuracies obtained by all 63 runs are shown in Table 4.2.

Table 4.2: Training accuracy (%) for all architecture–optimizer combinations.

Architecture	SGD	BGD	Momentum	NAG	AdaGrad	RMSPProp	Adam
3L-v1	100.00	89.93	100.00	100.00	97.91	93.97	99.72
3L-v2	100.00	89.28	100.00	100.00	97.03	96.86	99.57
3L-v3	100.00	89.10	100.00	100.00	97.00	96.76	99.76
4L-v1	100.00	89.82	100.00	100.00	98.38	95.13	99.85
4L-v2	100.00	89.52	99.99	100.00	97.76	98.03	99.95
4L-v3	100.00	90.00	100.00	100.00	98.30	97.94	99.70
5L-v1	99.98	90.65	99.83	100.00	96.37	93.82	99.66
5L-v2	99.99	90.15	100.00	100.00	96.44	94.73	99.26
5L-v3	99.99	89.96	100.00	99.99	98.45	93.54	99.89

The corresponding validation accuracies are given in Table 4.3.

Table 4.3: Validation accuracy (%) for all architecture–optimizer combinations.

Architecture	SGD	BGD	Momentum	NAG	AdaGrad	RMSPProp	Adam
3L-v1	98.05	89.54	98.21	98.13	96.13	93.28	97.76
3L-v2	97.97	88.93	98.13	98.23	95.76	95.92	97.92
3L-v3	97.71	88.91	98.31	98.21	95.52	95.60	97.58
4L-v1	97.97	89.78	98.18	98.21	96.79	94.26	97.55
4L-v2	97.79	89.80	98.31	98.16	96.44	96.55	97.76
4L-v3	97.89	89.09	98.39	98.31	96.44	96.42	97.65
5L-v1	97.81	90.12	97.81	98.16	95.39	92.89	97.81
5L-v2	97.87	90.28	97.89	98.16	95.28	93.97	96.71
5L-v3	98.08	89.30	98.39	98.31	96.47	92.94	97.55

Most SGD, momentum, and NAG runs achieve approximately 100% training accuracy. Adam also achieves training accuracy close to 100% across the

architectures. However, the corresponding validation accuracies are lower, demonstrating that training accuracy alone is insufficient for selecting the final model.

Momentum and NAG consistently achieve validation accuracies close to 98%. BGD performs substantially worse, with validation accuracy remaining near 89–90%. AdaGrad generally achieves approximately 95–96.5%, while RMSProp exhibits greater variation across architectures.

4.3 Aggregate Optimizer Comparison

The mean validation accuracy, mean convergence epochs, and mean wall-clock time across the nine architectures are shown in Table 4.4.

Table 4.4: Aggregate optimizer performance across the nine architectures.

Optimizer	Mean validation accuracy (%)	Mean epochs	Mean time (s)
SGD + NAG	98.21	18.3	235.37
SGD + Momentum	98.18	16.4	208.85
SGD	97.90	35.6	475.74
Adam	97.59	64.7	967.68
AdaGrad	96.02	559.9	1.85
RMSProp	94.65	128.9	0.38
Batch GD	89.53	1913.1	6.08

NAG obtains the highest mean validation accuracy of 98.21%, closely followed by SGD with momentum at 98.18%. However, SGD with momentum has the lowest mean convergence count at 16.4 epochs, compared with 18.3 epochs for NAG.

Vanilla SGD obtains a mean validation accuracy of 97.90% and requires 35.6 epochs on average. Adam achieves 97.59% mean validation accuracy with an average of 64.7 epochs. AdaGrad and RMSProp provide lower validation accuracies under the fixed experimental configuration.

The wall-clock times must be interpreted separately from convergence epochs because the optimizers use different batch sizes. Batch-based methods perform fewer parameter updates per epoch, whereas batch-size-one methods

perform one update per training sample. Therefore, a lower wall-clock time does not necessarily indicate faster convergence in terms of epochs.

4.4 Training-error Curves

The average training cross-entropy loss versus epoch is superimposed for all seven optimizers for each architecture.

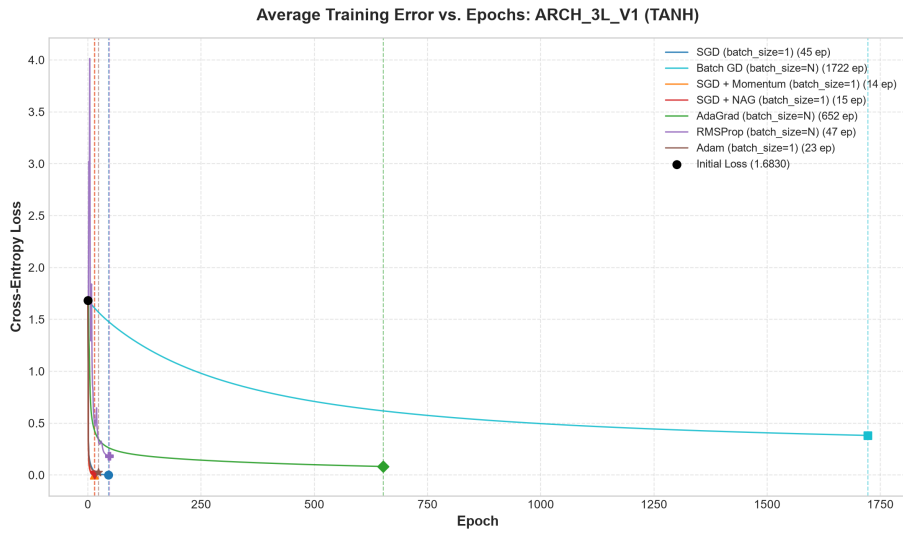


Figure 4.1: Superimposed average training cross-entropy loss for 3L-v1.

The training-error curves reinforce the numerical convergence results. Momentum and NAG exhibit rapid decreases in training loss and reach the convergence criterion within a small number of epochs. Vanilla SGD also reaches a low training loss but requires more epochs. BGD exhibits substantially slower convergence when measured in epochs.

The adaptive methods show different convergence behaviour depending on the architecture. These differences indicate that the choice of optimizer affects not only the final loss but also the trajectory followed during training.

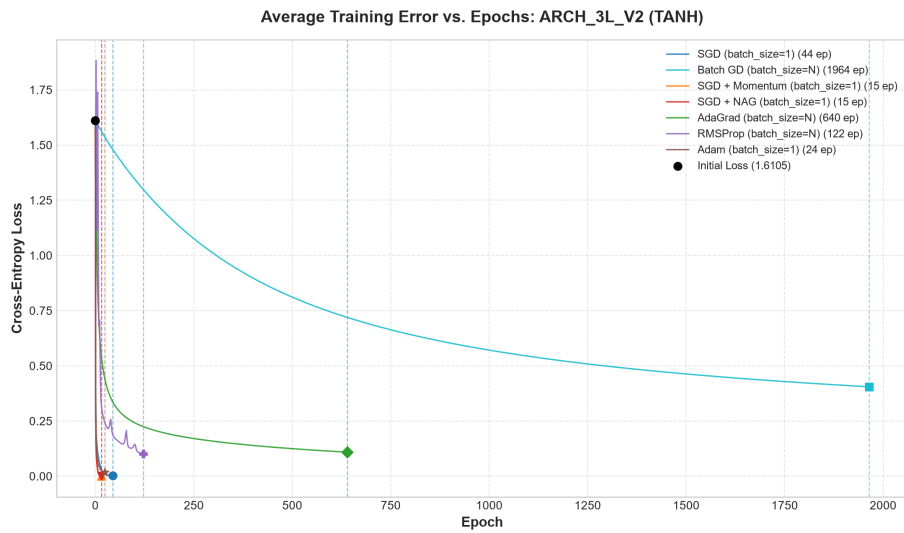


Figure 4.2: Superimposed average training cross-entropy loss for 3L-v2.

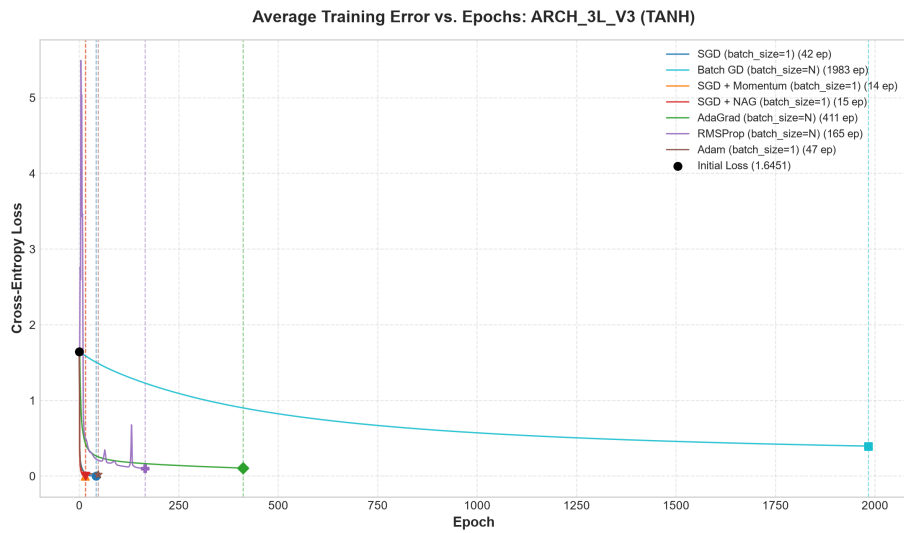


Figure 4.3: Superimposed average training cross-entropy loss for 3L-v3.

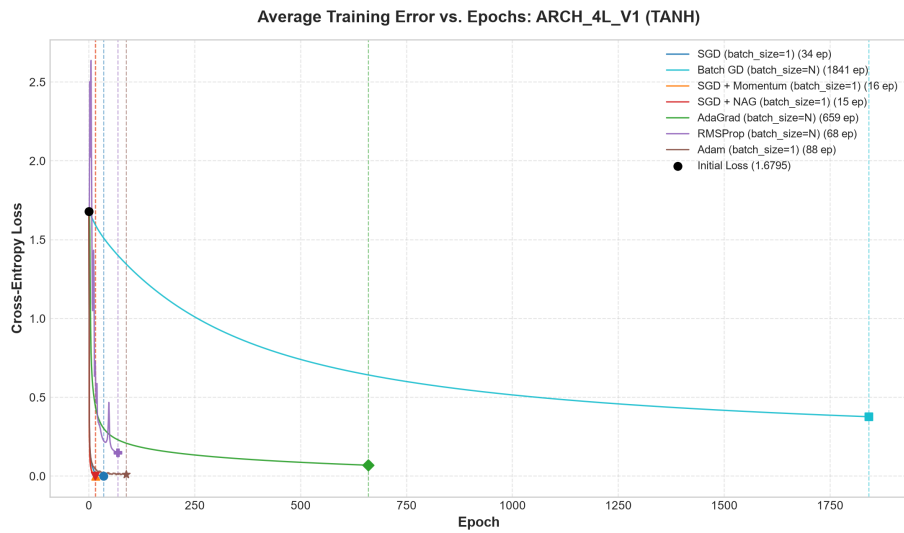


Figure 4.4: Superimposed average training cross-entropy loss for 4L-v1.

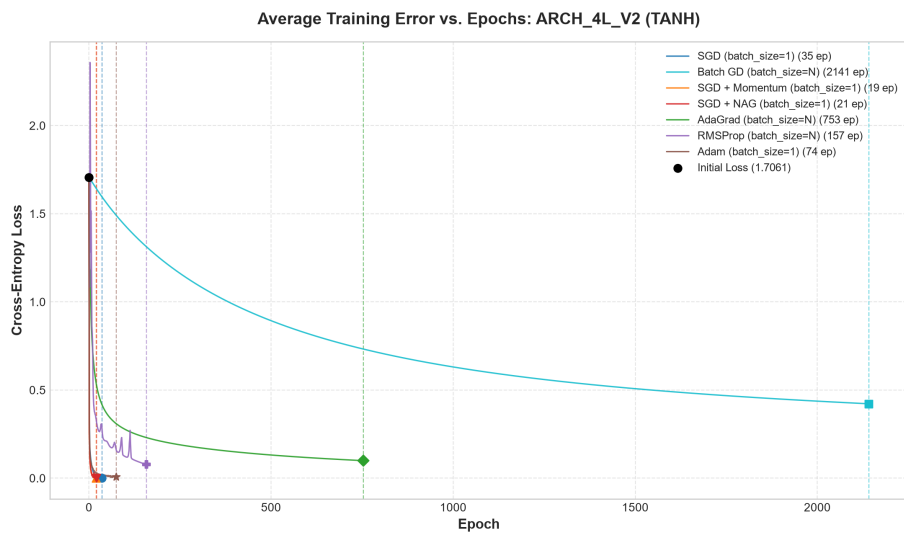


Figure 4.5: Superimposed average training cross-entropy loss for 4L-v2.

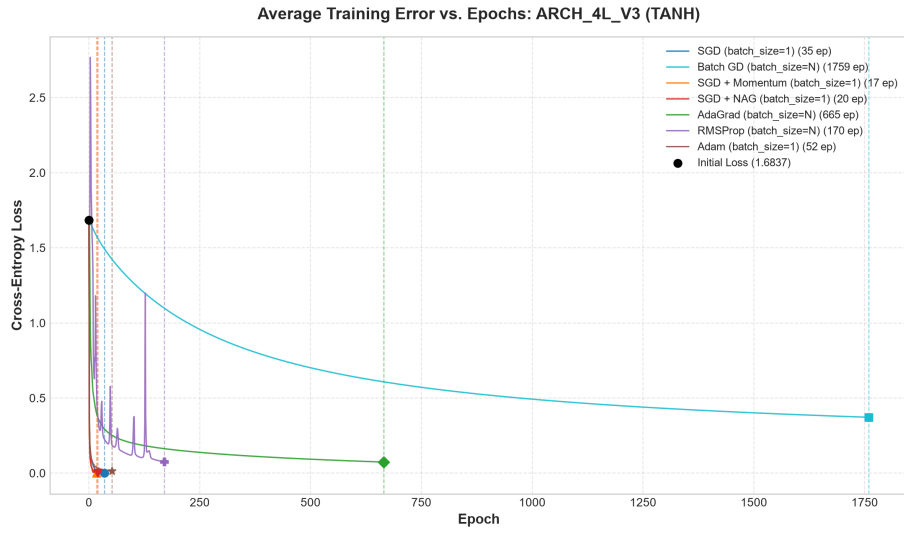


Figure 4.6: Superimposed average training cross-entropy loss for 4L-v3.

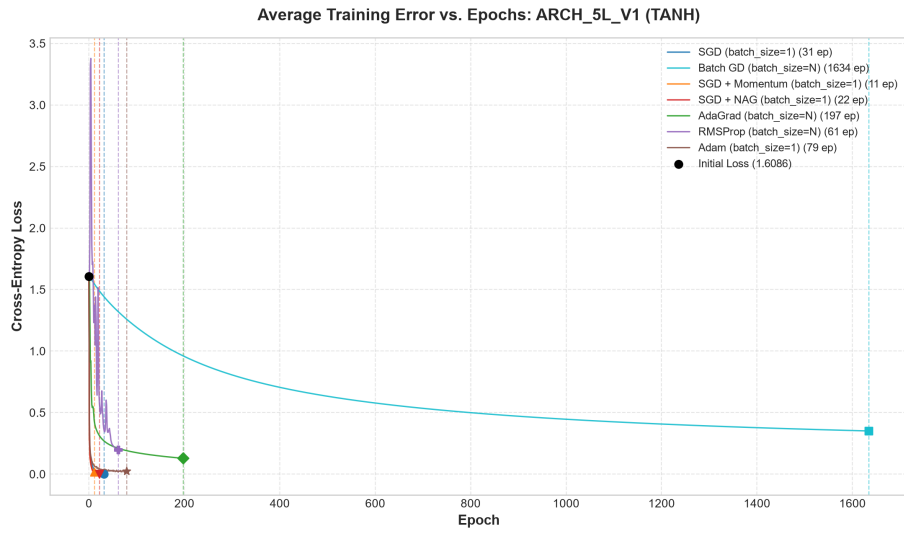


Figure 4.7: Superimposed average training cross-entropy loss for 5L-v1.

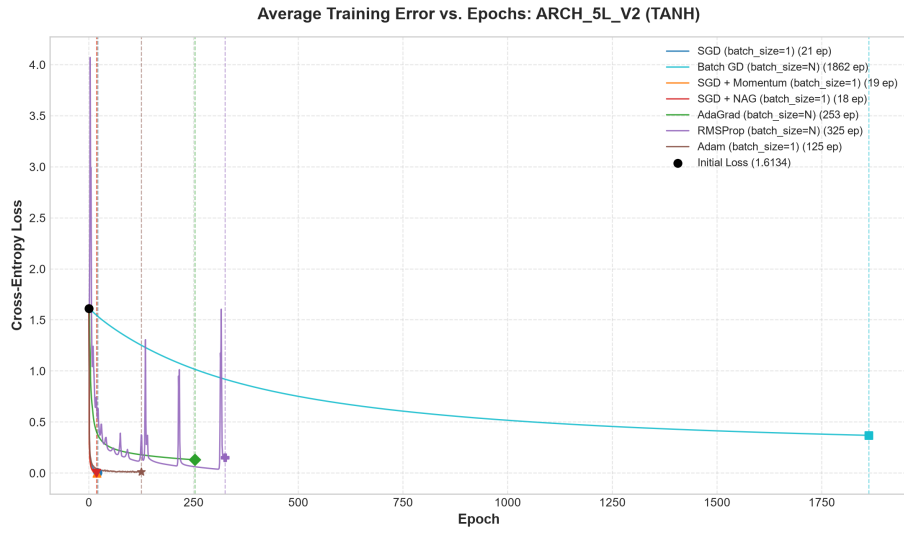


Figure 4.8: Superimposed average training cross-entropy loss for 5L-v2.

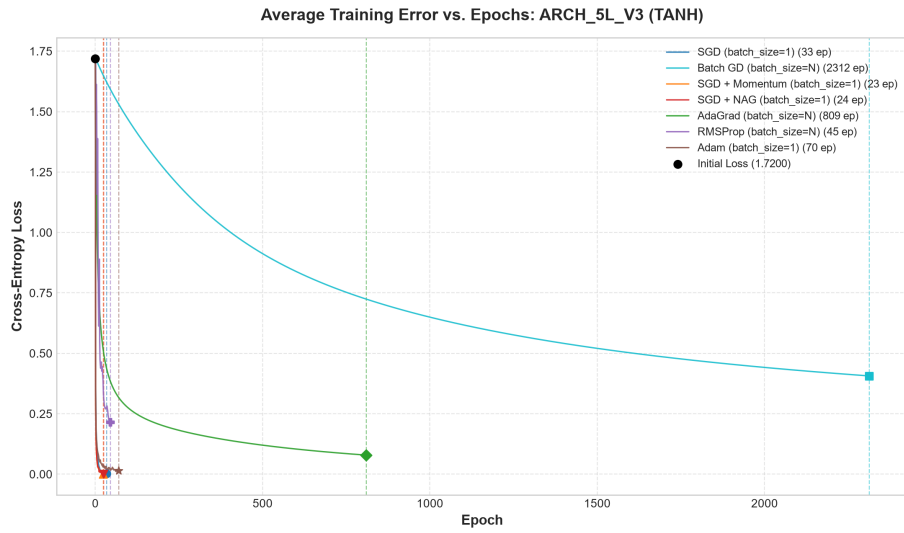


Figure 4.9: Superimposed average training cross-entropy loss for 5L-v3.

4.5 Effect of Depth and Width

Increasing the number of hidden layers does not result in a monotonic increase in validation accuracy. For SGD with momentum, the best validation accuracies for the three-, four-, and five-hidden-layer groups are 98.31%, 98.39%, and 98.39%, respectively.

Thus, adding a fifth hidden layer does not improve upon the best result obtained using four hidden layers.

The distribution of neurons across hidden layers also affects performance. The 4L-v3 architecture contains 441,477 trainable parameters and achieves 98.39% validation accuracy with momentum. In comparison, 4L-v1 contains 574,725 trainable parameters but does not exceed this validation accuracy.

Therefore, a larger parameter count does not necessarily result in better validation performance. The arrangement of the hidden-layer widths is also relevant to the resulting representation and optimization behaviour.

Chapter 5

Best Architecture and Test Evaluation

5.1 Model Selection

The highest validation accuracy obtained across the 63 experiments is

98.39%.

This value is achieved by both 4L-v3 and 5L-v3 when trained using SGD with momentum.

The two architectures are therefore tied on the primary model-selection criterion. To select a single model for final test evaluation, the tie is resolved using validation loss and convergence epochs. The 4L-v3 model has validation loss 0.0651 and converges in 17 epochs, whereas 5L-v3 has validation loss 0.0792 and converges in 23 epochs.

The selected model is therefore

4L-v3 + tanh + SGD with Momentum.

Its complete architecture is

$784 \rightarrow 384 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 5$.

5.2 Selected Model Performance

The performance of the selected model is summarized in Table 5.1.

Table 5.1: Performance of the selected 4L-v3 model.

Metric	Value
Initial training loss	1.683652
Final training loss	0.000258
Training accuracy	100.00%
Validation loss	0.065120
Validation accuracy	98.39%
Validation macro F1	98.39%
Convergence epochs	17

The model achieves 100% training accuracy and 98.39% validation accuracy. The difference between the two values indicates some degree of overfitting, although the validation performance remains high.

5.3 Selected Model Training Curve

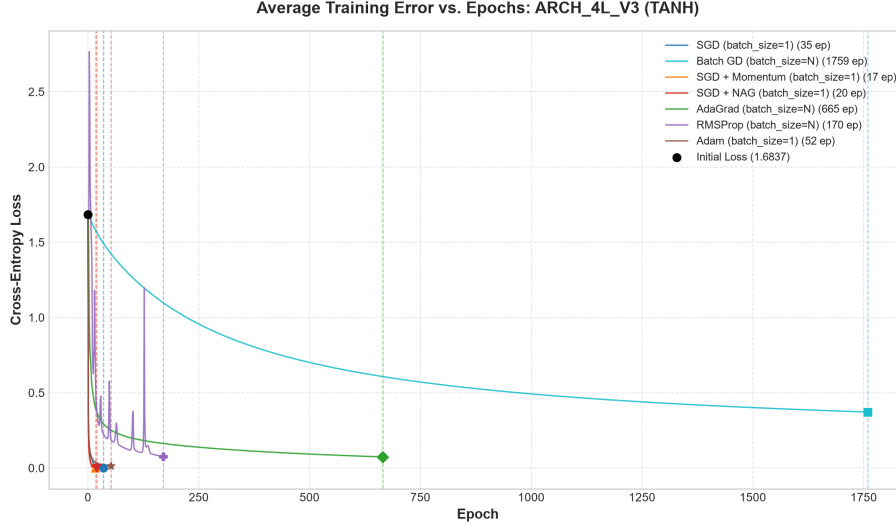


Figure 5.1: Optimizer comparison for the selected 4L-v3 architecture.

Figure 5.1 compares the average training cross-entropy loss across all seven optimizers for the selected 4L-v3 architecture. The black marker indicates the shared initial loss of approximately 1.6837, while the coloured dashed lines mark the epochs at which the stopping criterion is met. For the selected SGD-with-momentum run, the loss decreases to 0.000258 and the convergence criterion is satisfied at epoch 17. The very small final loss is consistent with the model’s 100% training accuracy, although training loss alone does not establish good performance of the model.

SGD with Momentum converges in the fewest epochs for this architecture, followed by NAG at 20 epochs, SGD at 35, and Adam at 52. In contrast, RMSProp, AdaGrad, and BGD require 170, 665, and 1,759 epochs, respectively. BGD shows a gradual decline, while AdaGrad decreases rapidly at first and then improves more slowly. RMSProp exhibits intermittent upward spikes before reaching a lower loss, indicating that its training trajectory is not monotonic.

These counts compare passes through the training set, not equal numbers

of parameter updates or equal wall-clock training times: the batch-size-one methods update after every sample, whereas the full-batch methods update once per epoch. Furthermore, the stopping rule measures the change in loss between consecutive epochs rather than requiring a common final loss. Thus, satisfying the same convergence criterion does not imply that all optimizers attain the same quality of fit.

5.4 Test-set Performance

The selected model is evaluated on the held-out test set containing 3,795 samples. The resulting test accuracy is

$$\text{Test accuracy} = 98.02\%.$$

The macro precision, macro recall, and macro F1-score are

$$\text{Macro precision} = 98.03\%,$$

$$\text{Macro recall} = 98.02\%,$$

and

$$\text{Macro F1} = 98.02\%.$$

The difference between validation and test accuracy is

$$98.39 - 98.02 = 0.37$$

percentage points. The small reduction indicates that the validation performance provides a close estimate of the performance observed on the held-out test set.

Test Confusion Matrix: ARCH_4L_V3 (TANH) - SGD + Momentum (batch_size=1)

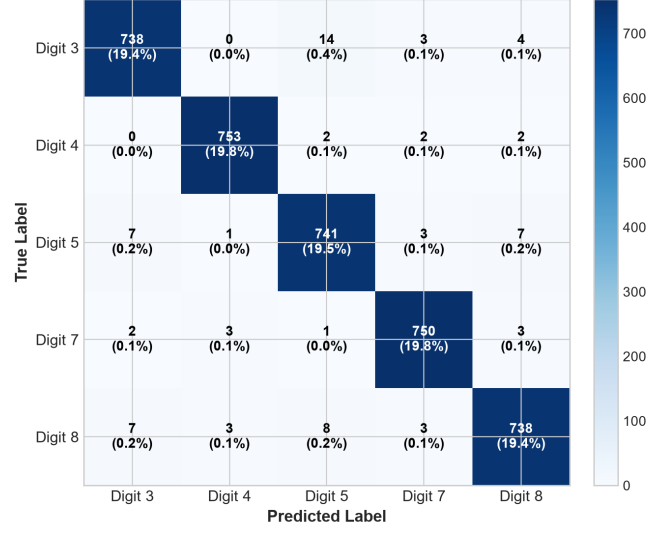


Figure 5.2: Test confusion matrix for the selected 4L-v3 model. Rows represent true classes and columns represent predicted classes in the order 3, 4, 5, 7, and 8.

5.5 Test Confusion Matrix

The numerical confusion matrix is

$$\begin{bmatrix} 738 & 0 & 14 & 3 & 4 \\ 0 & 753 & 2 & 2 & 2 \\ 7 & 1 & 741 & 3 & 7 \\ 2 & 3 & 1 & 750 & 3 \\ 7 & 3 & 8 & 3 & 738 \end{bmatrix}.$$

Each row contains the predictions for one true class, while each column corresponds to one predicted class. The diagonal entries therefore represent correctly classified samples.

The diagonal entries dominate for all five classes. The largest number of misclassifications occurs among the classes 3, 5, and 8, indicating that the remaining errors are concentrated among visually similar handwritten digits.

5.6 Training Confusion Matrix

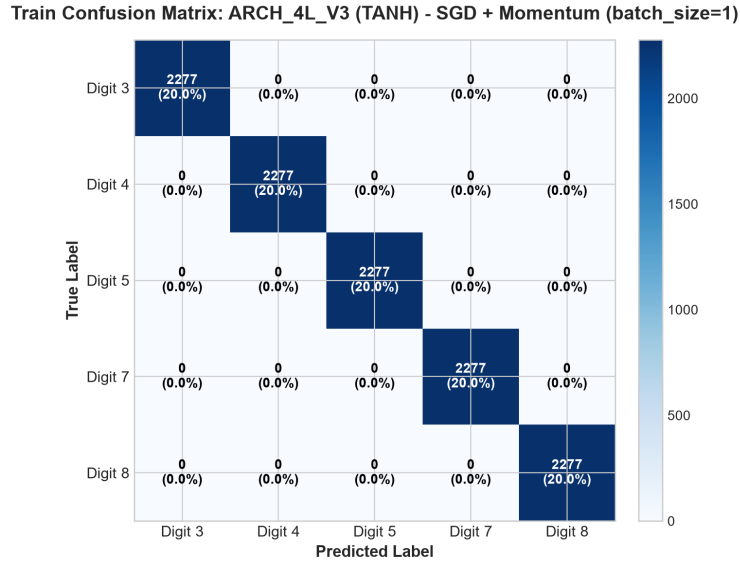


Figure 5.3: Training confusion matrix for the selected 4L-v3 model.

The training confusion matrix is diagonal, with 2,277 correctly classified samples for each of the five classes and no off-diagonal errors. This corresponds to 100% training accuracy.

The difference between the perfect training confusion matrix and the small number of test-set errors is consistent with the difference between training and test accuracy.

5.7 Per-class Test Performance

The precision, recall, and F1-score for each digit class are shown in Table 5.2.

Table 5.2: Per-class test performance of the selected model.

Digit	Precision (%)	Recall (%)	F1 (%)
3	97.88	97.23	97.55
4	99.08	99.21	99.14
5	96.74	97.63	97.18
7	98.55	98.81	98.68
8	97.88	97.23	97.55

Class 4 has the highest F1-score at 99.14%, while class 5 has the lowest at 97.18%. This is consistent with the confusion matrix, where class 5 participates in several of the remaining classification errors.

Chapter 6

Conclusion

This study compared seven optimization methods across nine FCNN architectures for five-class classification of handwritten digits. The experimental design controlled the initial weights within each architecture, used a common learning rate of 0.001, and employed the convergence criterion

$$|L_t - L_{t-1}| < 10^{-4}.$$

The results demonstrate that optimizer choice has a substantial effect on convergence behaviour. SGD with momentum and NAG consistently achieve validation accuracies close to 98% while converging in substantially fewer epochs than batch gradient descent and the adaptive methods under

the fixed experimental configuration.

NAG obtains the highest mean validation accuracy of 98.21%, while SGD with momentum obtains the lowest mean convergence count of 16.4 epochs. The highest individual validation accuracy is 98.39%, achieved by both 4L-v3 and 5L-v3 with SGD with momentum.

The 4L-v3 architecture is selected for final evaluation because it achieves the tied highest validation accuracy while also providing lower validation loss and faster convergence than 5L-v3. The selected model has the architecture

$$784 \rightarrow 384 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 5$$

and achieves 98.02% test accuracy and 98.02% macro F1-score.

The results also show that increasing network depth does not necessarily improve classification performance. The best four- and five-hidden-layer architectures both achieve 98.39% validation accuracy. Similarly, a larger number of trainable parameters does not guarantee improved validation performance.

Overall, under the experimental conditions considered in this study, momentum-based optimization provides the most favourable combination of convergence speed and classification performance. The results further indicate that optimizer selection and the distribution of network capacity can have a stronger practical effect than simply increasing the depth of the network.

References

- [1] Dileep. AD, “Essentials of Deep Neural Network: Better Optimization Methods for Backpropagation Algorithm,” CS601T: Deep Learning lecture slides, Indian Institute of Technology Dharwad, September 2026.
- [2] Y. Nesterov, “A method of solving a convex programming problem with convergence rate $O(1/k^2)$,” *Soviet Mathematics Doklady*, vol. 27, no. 2, pp. 372–376, 1983.
- [3] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, “On the importance of initialization and momentum in deep learning,” in *Proceedings of the 30th International Conference on Machine Learning*, pp. 1139–1147, 2013.
- [4] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011.
- [5] T. Tieleman and G. Hinton, “Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude,” *COURSERA: Neural Networks for Machine Learning*, 2012.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations*, 2015.
- [7] PyTorch Contributors, “torch.nn.init,” *PyTorch documentation*, accessed September 15, 2026.